

Final Exam

● Graded

Student

Christian Patrick Byrne

Total Points

91 / 110 pts

Question 1

Q1

4 / 4 pts

✓ - 0 pts Answer: Line 7

- 4 pts Incorrect

Question 2

Q2

4 / 8 pts

2.1 (a)

2 / 4 pts

- 0 pts Correct

- 2 pts i. should be $O(n^2)$

✓ - 2 pts ii. should be $n(n-1)/2$

2.2 (b)

2 / 4 pts

- 0 pts Correct

- 2 pts i should be $O(n \log n)$

✓ - 2 pts ii should be $(n-1) \cdot \log n$

Question 3

Q3

8 / 8 pts

3.1

(a)

2 / 2 pts

✓ - 0 pts Correct. (i)

- 2 pts Incorrect.

3.2

(b)

2 / 2 pts

✓ - 0 pts Correct. (iv)

- 2 pts Incorrect.

3.3

(c)

2 / 2 pts

✓ - 0 pts Correct. (iv)

- 2 pts Incorrect.

3.4

(d)

2 / 2 pts

✓ - 0 pts Correct. (ii)

- 2 pts Incorrect.

Question 4

Q4

11 / 12 pts

4.1

(a)

1 / 1 pt

✓ - 0 pts Correct

- 1 pt Incorrect; Algorithm / Asymptotic Analysis

4.2

(b)

1 / 1 pt

✓ - 0 pts Correct

- 1 pt Ω, ω

4.3

(c)

1 / 1 pt

✓ - 0 pts Correct

- 1 pt Incorrect; n^3

4.4

(d)

1 / 1 pt

✓ - 0 pts Correct

- 1 pt Incorrect;
Simple, Complex

4.5

(e)

1 / 1 pt

✓ - 0 pts Correct

- 1 pt Key Comparisons, Data Movements

4.6

(f)

1 / 1 pt

✓ - 0 pts Correct

- 1 pt Incorrect; 16

4.7

(g)

1 / 1 pt

✓ - 0 pts Correct

- 1 pt Incorrect; All-Pairs Shortest Path (APSP)

4.8

(h)

1 / 1 pt

✓ - 0 pts Correct

- 1 pt Incorrect; Dynamic Programming

4.9

(i)

1 / 1 pt

✓ - 0 pts Correct

- 1 pt Incorrect; Backtracking

4.10 (j)

1 / 1 pt

✓ - 0 pts Correct

- 1 pt Correct Answer: Principle of Optimality or Greedy Algorithm family

4.11 (k)

0 / 1 pt

- 0 pts Correct

✓ - 1 pt Incorrect / Blank

Vertex Cover

4.12 (l)

1 / 1 pt

✓ - 0 pts Correct

- 1 pt Incorrect

NP

Question 5

Q5

2 / 6 pts

- 0 pts Correct. The root is K. Its children are G and R. G's children are D and I. R's children are N and TY. D's children are AC and F. I's children are H and J. N's children are M and P. And, the three children off of the TY node are S, UW, and Z.

- 1 pt Minor error. Compare your tree to the above description.

- 3 pts Your tree isn't a legal 2-3 tree because some of your internal nodes either have too many or two few children for the number of data values they contain.

- 3 pts Your tree is unbalanced, probably because you didn't promote all the way to (and through) the root.

✓ - 4 pts Insertion isn't the opposite of deletion; we do not distribute to existing neighbors, we promote up when a node is over-full. I is promoted to the DG node, G is promoted to KR, and K is promoted to a new parent.

- 5 pts You inserted more or less like you were inserting into a binary search tree. 2-3 trees grow up, not down.

- 6 pts Blank or substantially incorrect. See the "Correct" item above for the test description.

Question 6

Q6

4 / 6 pts

- 2 pts `countLeaves` returns void--should return total number of leaf nodes

- 1 pt Minor error in `if (root == null) return 0;`

✓ - 2 pts Major error in `if (root == null) return 0;`

- 1 pt Minor error in `if ((root.lchild == null) && (root.rchild == null)) {
return 1;
}`

- 2 pts Major error in `if ((root.lchild == null) && (root.rchild == null)) {
return 1;
}`

- 1 pt Minor error in `return countLeaves(root.lchild) + countLeaves(root.rchild);`

- 2 pts Major error in `return countLeaves(root.lchild) + countLeaves(root.rchild);`

- 6 pts Incorrect or Blank;

```
public int countLeaves(TreeNode root)
{
    if (root == null) {
        return 0;
    }
    if ((root.lchild == null) && (root.rchild == null)) {
        return 1;
    }
    return countLeaves(root.lchild) + countLeaves(root.rchild);
}
```

- 3 pts Major error in recursion

- 0 pts Correct

- 1 pt Minor error

- 3 pts Major error

3

What if node is null?

Question 7

Q7

13 / 13 pts

7.1

(a)

4 / 4 pts

✓ - 0 pts Correct. The two recurrences are $T(n) = T(n/9) + 12$ and $T(n) = T(n/27) + 18$.

- 2 pts About half right. See the answers in the "Correct" criteria item, above.

- 4 pts Blank or substantially incorrect response. See the answers in the "Correct" criteria item, above.

7.2

(b)

2 / 2 pts

✓ - 0 pts Correct. $T(n) = T(n/3^a) + 6a$

- 1 pt You missed either the 3^a or the $6a$.

- 2 pts Blank or substantially incorrect response.

7.3

(c)

4 / 4 pts

✓ - 0 pts Correct. $T(n) = 6 \log_3 n + 2$

- 1 pt You don't have a log of n function.

- 1 pt Your log isn't base 3

- 1 pt You didn't multiply the log by 6.

- 1 pt You don't have the + 2.

- 4 pts Blank or significantly incorrect response.

7.4

(d)

3 / 3 pts

✓ - 0 pts Correct

- 1 pt $T(1) = 2$

- 1 pt $T(3) = 8$

- 1 pt $T(9) = 14$

Question 8

Q8

10 / 10 pts

8.1 (a)

0 / 0 pts

✓ - 0 pts Correct

i)

✓ - 0 pts Chose i) and correct;

Basis step: $n = 1$ $T(1) = 2$ by the recurrence, and $T(1) = 6 \log_3 1 + 2 = 0 + 2 = 2$ from the closed-form solution. OK.Inductive step: If $T(n) = 6 \log_3 n + 2$, then $T(3n) = 6 \log_3(3n) + 2$.

$$\begin{aligned}
 T(3n) &= T(n) + 6 && \text{By the Recurrence} \\
 &= [6 \log_3 n + 2] + 6 && \text{By the I.H.} \\
 &= 6 \log_3 n + 6 + 2 && \text{Commutativity} = 6(\log_3 n + 1) + 2 && \text{Factor out the 6} \\
 &= 6(\log_3 n + \log_3 3) + 2 && \log_b b = 1 \\
 &= 6 \log_3(3n) + 2 && \text{Sum of logs of same base is product}
 \end{aligned}$$

Q.E.D.

- 2 pts Did not plug into closed form in basis step
- 3 pts Missing basis step
- 1 pt Minor error in inductive step
- 1 pt Did not correctly state where inductive hypothesis is applied
- 3 pts Error in inductive step
- 4 pts Tried to step from k to $k + 1$ and did not reach conclusion properly
- 5 pts Major error in inductive step
- 7 pts Missing inductive step

ii)

- 5 pts Chose ii) (max 5/10 points);

Basis step: $n = 1$ $H(1) = 4$ by the recurrence, and $H(1) = \frac{7(1-1)}{2} + 4 = 0 + 4 = 4$ from the closed-form solution. OK.Inductive step: If $H(n) = \frac{7(n-1)}{2} + 4$, then $H(n+2) = \frac{7(n+1)}{2} + 4$.

$$\begin{aligned}
 H(n+2) &= H(n) + 7 && \text{By the Recurrence} \\
 &= \left[\frac{7(n-1)}{2} + 4 \right] + 7 && \text{By the I.H.} \\
 &= \frac{7(n-1)}{2} + 7 + 4 && \text{Commutativity} = \frac{7n-7}{2} + \frac{14}{2} + 4 \\
 &= \frac{7n+7}{2} + 4 \\
 &= \frac{7(n+1)}{2} + 4
 \end{aligned}$$

Q.E.D.

- 1 pt Did not plug into closed form in basis step
- 1 pt Error in basis step
- 2 pts Missing basis step
- 1 pt Did not correctly state where inductive hypothesis is applied

- 1 pt Minor error in inductive step
- 2 pts Tried to step from k to $k + 1$ and did not reach conclusion properly
- 2 pts Major error in inductive step
- 3 pts Missing inductive step

- 10 pts Blank/Incorrect

Question 9

Q9

4 / 6 pts

- 0 pts Correct

✓ - 2 pts minor error in a pass, or you had the right idea but did the passes in the reverse order.

- 3 pts Incorrect # of passes, or a good start that fell apart in a later pass.

- 6 pts blank/incorrect. There are four queues, 0, 1, 2, and 3. In the first pass, we place values into those queues by the digit in the units position. Queue 0: 30, 10. Q1: 11. Q2: 2. Q3: 123, 33. In the second pass, remove from the queues in order (0-to-3) and re-insert based on the ten's position. Q0: 2. Q1: 10, 11. Q2: 123. Q3: 30, 33. Third pass, hundred's position. Q0: 2, 10, 11, 30, 33. Q1: 123. The longest number has three digits, so there are three passes.

Question 10

Q10

6 / 7 pts

10.1 (a)

3 / 3 pts

✓ - 0 pts Correct

- 1 pt Missing detail about searching linearly

- 1 pt Minor error; Missing detail

- 1 pt Only provided the hash function, and not the definition.

- 3 pts Incorrect / Blank

When a collision is detected at index i , we instead try to place the item in location $(i + 1) \% n$, where n is the number of 'slots' in the table. If that location is occupied, we try $(i + 2) \% n$, etc.

10.2 (b)

3 / 4 pts

- 0 pts Correct

✓ - 1 pt Missing discussion about trying to avoid collisions

- 2 pts Missing detail; or major error

- 4 pts Incorrect / Blank;

The image of a digest function, a message digest, should be unlikely to be the image produced by any other input value. That is, collisions should be hard to achieve, because you don't want it to be easy for an attacker to be able to trick us into believing that a false message (or software archive or...) is the true message.

Question 11

Q11

11 / 16 pts

11.1 (a)

2 / 6 pts

- 0 pts Correct

- 1 pt minor error in the order

✓ - 2 pts Incorrect Fringe list

✓ - 2 pts Incorrect Known list

- 6 pts blank incorrect

1 Why is D back?

2 B can't be ahead of D

11.2 (b)

3 / 3 pts

✓ - 0 pts Correct

- 2 pts Answered no

- 1 pt Failed to explain why Dijkstra is greedy - we always grab the vertex with the smallest available total cost without considering the long-term cost.

11.3 (c)

6 / 7 pts

- 1 pt i. A, C, & G

- 1 pt ii. Yes. (No self loops)

- 1 pt iii. No. (D-A-E-D)

- 1 pt iv. 4 (3 in, 1 out)

✓ - 1 pt v. Yes

- 1 pt vi. No

- 1 pt vii. No

- 0 pts Correct

Question 12

Q12

12 / 12 pts

12.1 (a)

6 / 6 pts

✓ - 0 pts Correct

- 2 pts bestcase: $O(n \log n)$
- 2 pts worst case: $O(n^2)$
- 2 pts average case: $O(n \log n)$

12.2 (b)

6 / 6 pts

✓ - 0 pts Correct

- 1 pt i. Unchanged
- 1 pt ii. Improved
- 1 pt iii. Unchanged
- 1 pt i. Did not explain / insufficient explanation
Correct: The tree will be acceptably busy either way.
- 1 pt ii. Did not explain / insufficient explanation
Correct: Building an AVL tree is more work, but nowhere near $O(n^2)$.
- 1 pt iii. Did not explain / insufficient explanation
Correct:
The trees are similarly bushy.

Question 13

Q13

2 / 2 pts

✓ - 0 pts Correct. One way to explain it is to think about sorting algorithms. We have many of those, each with their own pros and cons. The same is true of heaps, just in the data structure sense.

- 1 pt Not quite there. See text box.
- 2 pts Blank or substantially incorrect. See the "Correct" criteria item for one possible answer.

Final Exam

December 18, 2024

Your Name
(printed!):

Christian Byrne

UA
NetID:

cbyrne

Directions

1. **DO NOT OPEN THIS EXAM UNTIL INSTRUCTED TO DO SO!**
2. Print your name and NetID in the blanks above. The closer your name matches the form of your name as Gradescope knows it, the better.
3. This exam is closed book, closed notes, closed neighbor, open mind. *Keep your eyes on your own exam!*
4. The use of electronic devices, including but not limited to calculators, cell phones, earbuds, and smart watches, is NOT permitted on this exam.
5. Be sure that your answers fit within the spaces provided. If you need scratch paper, use the last sheet.
6. Make your answers as precise and concise and to the point as possible, while still answering the questions asked.
7. Show your work, where appropriate, for potential partial credit. Vague, incomplete, and/or ambiguous answers will not receive full credit.
8. If you have any questions, raise your hand. We'll (try to!) come to you.

Page	Points
1	12
2	8
3	12
4	12
5	13

Page	Points
6	10
7	13
8	16
9	14
Total	110

$$d^n = 1 \quad d \in \mathbb{R}, d \neq 0, n \in \mathbb{Z}^+$$

$$d^{k+2}$$

1. (4 points) There is a flaw in the reasoning of the following proof by induction. On which line is the flaw located?

Conjecture: $d^n = 1$, where d is a non-zero real number, and n is a non-negative integer.

1 Proof (strong induction):
 2 Basis Step: Let $n=0$. $d^0 = 1$.
 3 Inductive Step: If $d^i = 1 \forall i$ such that $0 \leq i \leq k$, then $d^{k+1} = 1$.
 4
 5 d^{k+1}
 6 $= d^{k+2} / d^1$
 7 $= (d^k \cdot d^k) / d^1$
 8 $= (1 \cdot 1) / 1$
 9 $= 1$
 10
 11 Therefore, $d^n = 1$, where d is a non-zero real number,
 12 and n is a non-negative integer.

$$d^{k+2} = d^k \cdot d^2$$

$$d^{k+2} \neq d^k \cdot d^k = d^{2k}$$

On which line of the proof is the flaw located? Circle your answer from the options listed below.

Line 2 Line 3 Line 6 Line 7

2. (8 points) Step-Counting and $\Theta()$. For each of the following pseudocode fragments, what is the $\Theta()$ of its execution time, and what is the final value of sum in terms of n , assuming that n is a positive power of 2 greater than 1? In each case, circle your answer from the listed options.

(a)

```
1 sum ← 0
2 for i ← 1 through n-1:
3   for j ← 1 through i:
4     sum ← sum + 1
```

$$\left(\frac{n(n+1)}{2} \right) \rightarrow n^2$$

$$n = 2^k, n > 1$$

- i. The $\Theta()$ asymptotic complexity of this code is:

$\Theta(n^2)$ $\Theta(n \cdot \log_2 n)$ $\Theta(n)$ $\Theta(\log_2 n)$

Sum = 0
 for i in range(1, n):
 for j in range(1, i):
 sum += 1

add 1
 add 2
 add 3
 add 4
 ...
 add n-1

- ii. The final value of sum is:

n^2 $n(n+1)/2$ $n(n-1)/2$ $n-1$

(b)

```
1 sum ← 0
2 for i ← 1 through n-1:
3   j ← n
4   while j > 1:
5     sum ← sum + 1
6     j ← j / 2
```

Sum = 0
 for i in range(1, n):
 for j in range(1, 2, 1):
 sum++
 j /= 2

Sum 1...n
 $K = \log_2 n + 1$
 add 1 K times

- i. The $\Theta()$ asymptotic complexity of this code is:

$\Theta(\log_2 n)$ $\Theta(n \log_2 n)$ $\Theta(n)$ $\Theta(n^2)$

$\downarrow$ for
 $n \log_2 n + 1$
 $\in n \log_2 n$

- ii. The final value of sum is:

$(n-1) \cdot \log_2 n$ $n \cdot \log_2 n$ $n + \log_2 n$ $\log_2 n$

5
 1 2 3 4

3. (8 points) Algorithm Analysis. The parts of this question refer to the Anonymous Algorithm shown below, in pseudocode. Input to the algorithm is an undirected graph $G = (V, E)$, a source vertex s , and G 's representation using an adjacency list $\text{adjList}[]$. Other data structures used are arrays $a[]$ and $b[]$, and an initially empty vertex stack S . A sample graph and its adjacency list representation are provided for your convenience.

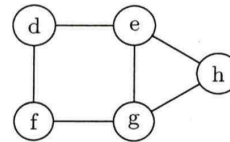
ANONYMOUS ALGORITHM

```

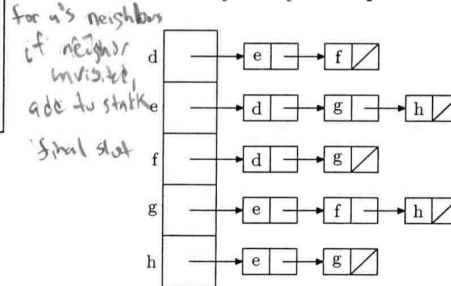
1  for each  $v \in V$ :
2     $a[v] \leftarrow \text{false}$ 
3     $b[v] \leftarrow \text{nil}$ 
4   $S.\text{push}(s)$ 
5  while  $S.\text{size}() \neq 0$ :
6     $u \leftarrow S.\text{pop}()$ 
7    if  $a[u]$  holds false:
8       $a[u] \leftarrow \text{true}$ 
9      for each  $n$  in  $\text{adjList}[u]$ :
10       if  $a[n]$  holds false:
11          $S.\text{push}(n)$ 
12          $b[n] \leftarrow u$ 

```

A sample graph:



And its adjacency list representation:



input
 $G = (V, E) \sim \text{adjList}[]$
 source vertex
 s
 $a[]$
 $b[]$
 $S = \text{stack vertex}$

For each question below, circle the label (i, ii, iii, or iv) of the best available answer.

- (a) How does the content of array $a[]$ support the execution of Anonymous Algorithm?
- ☒ i. The array keeps track of which vertices have been visited.
 - ii. The array provides confirmation of the quantity of vertices considered.
 - iii. At the end, a remaining 'false' value indicates that the graph is acyclic.
 - iv. At the end, the number of 'false' values equals the number of untraveled edges.
- (b) After the algorithm has been executed, what information about G does the array $b[]$ contain?
- i. For each vertex, the vertex that is closest to it.
 - ii. The path distances from s to each vertex
 - iii. For each edge (x, y) , the vertex $(x \text{ or } y)$ that was examined last.
 - ☒ iv. The last vertex from which each vertex was reached
- (c) What is the actual name of this algorithm?
- i. Prim's Minimal Cost Spanning Tree Algorithm
 - ☒ ii. Kruskal's Minimal Cost Spanning Tree Algorithm
 - iii. Breadth-First Search (BFS)
 - ☒ iv. Depth-First Search (DFS)
- (d) What is the $\Theta()$ approximation of the execution time of this algorithm?
- i. $\Theta(|V|^2)$
 - ☒ ii. $\Theta(|V| + |E|)$
 - iii. $\Theta(|E| \log_2 |V|)$
 - iv. $\Theta(|E| + |V| \log_2 |V|)$

4. (12 points) Miscellaneous "Fill In The Blank" questions.

$a \text{ vs. } b^d$
 $T(n) = a \cdot T(\frac{n}{b}) + c \cdot d^n$
 $n = b^k$

* (a) asymptotic analysis is the study of the work performed (or space consumed) by an algorithm as the size of the problem grows.

(b) In algorithm analysis, the two Greek letters Ω and ω label the two lower bounds, the 'mirror images' of $O()$ and $o()$.

(c) According to the Master Theorem, $T(n) = 8T(n/2) + 10n^2$ is $O(\underline{n^{\log_2 8}})$.

(d) Simple and Complex are the English names of the two complexity groups to which sorting algorithms typically belong.

(e) Sorting algorithms are placed into one of those two complexity groups (see previous question) due to how efficiently they use two types of operations. What are those two types of operations?

Key Comparisons and Data Movements

(f) 16 is the number of elements in each run (except perhaps the last) of Two-Way External Merge Sort after four passes, assuming that initially each sorted run contains just one element.

(g) The Floyd-Warshall Algorithm solves the All-pairs-shortest-path problem

(h) Algorithms in the Dynamic Programming algorithm "family" construct tables of answers to sub-problems whose solutions were computed in the process of attempting to solve a larger problem.

(i) Algorithms in the Backtracking algorithm "family," unlike those from the Greedy family, are able to return to a prior decision point to make a new choice.

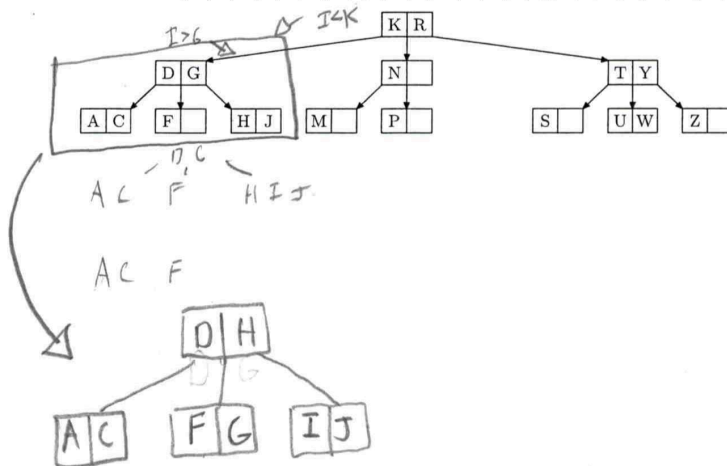
(j) "In an optimal sequence of decisions / choices, every subsequence must also be optimal." This sentence is the description of the ...

Principle of optimality

(k) The solutions to the Minimum Spanning Tree problem are minimal sets of a graph's vertices such that all of the graph's edges are defined using one or both of those vertices.

(l) NP is the common abbreviation of the phrase "Non-Deterministic Polynomial Time."

5. (6 points) Insert 'I' into the following 2-3 tree and show the resulting tree. You need only draw the portions of the tree that change as a result of the insertion. *Not a native English speaker?* The letters of the alphabet are ordered A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S,T,U,V,W,X,Y,Z.



TreeNode {
data
lchild
rchild

6. (6 points) Write a complete, syntactically-correct, recursive Java method named `countLeaves` that accepts a pointer/reference to the root node of a binary tree and returns the total number of leaf nodes in the tree. Assume that you are given a `TreeNode` class that contains three public fields: A data field named `data`, a left subtree pointer field named `lchild`, and a right subtree pointer field named `rchild`.

```
public int countLeaves(TreeNode node) {
    if (!node.lchild && !node.rchild) // Brackets not necessary for this if statement
        return 1;
    if (!node.lchild)
        return countLeaves(node.rchild);
    if (!node.rchild)
        return countLeaves(node.lchild);
    return countLeaves(node.rchild) + countLeaves(node.lchild);
}
```


$$n \geq 1 \quad T(n) = T\left(\frac{n}{3}\right) + 6$$

$$n = 3^i \quad T(1) = 2$$

7. (13 points). Consider the recurrence relation $T(n) = T(n/3) + 6$, with an initial condition of $T(1) = 2$, and the "Find the Pattern" (a.k.a. Backward Substitution) approach to solving recurrences. Assume that $n \geq 1$ and is a power of three.

- (a) Determine two equivalent recurrences for $T(n)$, the first in terms of $T(n/9)$ and the second in terms of $T(n/27)$.

$$T(n) = T\left(\frac{n}{9}\right) + 12$$

$$T(n) = T\left(\frac{n}{27}\right) + 18$$

$$T\left(\frac{n}{3}\right) = T\left(\frac{n/3}{3}\right) + 6 = T\left(\frac{n}{9}\right) + 6$$

$$T\left(\frac{n}{9}\right) = T\left(\frac{n/9}{3}\right) + 6 = T\left(\frac{n}{27}\right) + 6$$

$$T(n) = (T\left(\frac{n}{9}\right) + 6) + 6 = T\left(\frac{n}{9}\right) + 12$$

$$T(n) = (T\left(\frac{n}{27}\right) + 6) + 6 + 6 = T\left(\frac{n}{27}\right) + 18$$

- (b) "Find the pattern" of $T(n)$. That is, what is the general expression for $T(n)$ in terms of $T(n/3^a)$?

$$T(n) = T\left(\frac{n}{3^a}\right) + 6a$$

$$\frac{n}{3^a} = 1 \Rightarrow n = 3^a \Rightarrow a = \log_3 n$$

$$T\left(\frac{n}{3^{\log_3 n}}\right) + 6 \cdot \log_3 n =$$

$$T(1) + 6 \log_3 n =$$

$$2 + 6 \log_3 n =$$

$$2 + 6 \log_3 n$$

- (c) Create the equivalent closed-form expression for $T(n)$.

$$T(n) = 2 + 6 \cdot \log_3 n$$

- (d) Using your closed-form expression, evaluate it on the inputs 1, 3, and 9, and give the three results.

$$T(1) = 2$$

$$T(3) = 8$$

$$T(9) = 14$$

$$2 + 6 \log_3 1$$

$$2 + 0$$

$$2$$

$$2 + 6 \cdot \log_3 3$$

$$2 + 6 \cdot 1$$

$$2 + 6$$

$$8$$

$$2 + 6 \cdot \log_3 9$$

$$2 + 6 \cdot 2$$

$$2 + 12$$

$$14$$

8. (10 points) Proving closed-forms of recurrence relations.

(a) Please copy your closed-form solution from Question 7(c) onto the line provided below:

$$\underline{T(n) = 2 + 6 \cdot \log_3 n}$$

(b) You have a choice: Either prove inductively (i) that your closed-form solution is equivalent to $T(n) = T(n/3) + 6$ with $T(1) = 2$, or, for a maximum of half-credit on this question, (ii) that $H(n) = \frac{7(n-1)}{2} + 4$ is the closed-form solution to the recurrence $H(n) = T(n-2) + 7$, where $H(1) = 4$ and n is odd. Suggestion: Sketch out your answer(s) on the scratch paper before you make your decision.

Circle the label of your choice: (i) (ii)

Proof that closed-form solution $T(n) = 2 + 6 \cdot \log_3 n \equiv T(n) = T(\frac{n}{3}) + 6$
with $T(1) = 2$

Inductive Hypothesis: $T(n) = 2 + 6 \cdot \log_3 n$

Basis: Let $n=1$, Then:

$$T(1) = 2 + 6 \cdot \log_3 1 = 2 + 6 \cdot 0 = 2$$

$$T(1) = 2 \quad \text{Good}$$

Inductive Step:

WTP: If $T(K) = 2 + 6 \cdot \log_3 K$ for some $K \geq 1$ s.t. $K = 3^i$ for some $i \in \mathbb{Z}$ is equivalent to $T(K) = T(\frac{K}{3}) + 6$ then

$T(3K) = 2 + 6 \cdot \log_3 (3K)$ is also equivalent.

$$T(3K) = T(\frac{3K}{3}) + 6 = T(K) + 6 = 2 + 6 \cdot \log_3 K + 6 \quad (\text{By the IH})$$

$$= 2 + 6 (\log_3 K + 1) = 2 + 6 (\log_3 K + \log_3 3)$$

$$= 2 + 6 (\log_3 3K) = 2 + 6 \cdot \log_3 3K \quad \text{which is the exact formula we wanted to show the equivalence for.}$$

Therefore, if the closed form is equivalent for K , it is also equivalent for $3K$.
and by the principle of mathematical induction, $T(n) = 2 + 6 \log_3 n$ (closed form)
is equivalent to $T(n) = T(\frac{n}{3}) + 6$ with $T(1) = 2$ $\square$

9. (6 points) Use Radix Sort to arrange the following collection of Base 4 values into ascending order. Show the state of the sort after each pass (including the final pass).

Pass 1	123 030 011 010 033 002	Pass 2	Pass 3	Final
0 30, 10		0 2	0 2, 10, 11, 30, 33	0 2, 10, 11, 30, 33, 123
1 11		1 10, 11	1 123	1
2 2		2 123	2	2
3 123, 33		3 30, 33	3	3

Sorted array in first bucket

10. (7 points) Hashing.

(a) Define: linear probing

Linear probing is a collision resolution strategy for resolving collisions in a hash table — It works by, when a collision occurs, attempting the next slot in the table sequentially and repeatedly until an open slot is found. It may be implemented with a wrapper function like $Lp(K, i) = (h(K) + i) \% m$ where K is the key, i is the increment, $h()$ is hashing fun, and m is table size.

(b) Program #4 ("They did the MASH...") asked you to implement a digest function. How are the purposes of digest and hash functions similar?

There are multiple ways hashing and digest functions may be seen as similar. One such way is by nature of both encoding a value into a key and possessing the ability to decode the key back to the initial value homogeneously. Digest functions are encoding for the purpose of cryptography and hashing for the purpose of hashing but in essence those two purposes share the underlying property of homogeneously encoding and decoding a value — "homogeneously" entailing a fixed codomain.

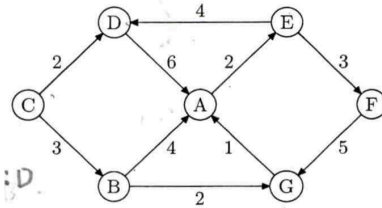
11. (16 points) More Graphs.

- (a) Consider this digraph. Perform Dijkstra's Algorithm on it, starting with vertex C. Use the provided table to keep track of the known vertices and the content of the fringe list as the algorithm proceeds, including the current costs to reach each vertex in the table. The first line is provided to get you started.

I don't remember exact notation for this table setup

Known	Fringe
$C_{0,-}$	$D_{2,C}, B_{3,C}$
$C_{0,-}, B_{3,C}, D_{2,C}$	$A_{8,D}$
$C_{0,-}, B_{3,C}, D_{2,C}, A_{8,D}$	$F_{7,B}, G_{5,B}$
$C_{0,-}, B_{3,C}, D_{2,C}, G_{5,B}, A_{8,D}$	$A_{6,G}$
$C_{0,-}, B_{3,C}, D_{2,C}, G_{5,B}, A_{6,G}$	$E_{8,A}$
$C_{0,-}, B_{3,C}, D_{2,C}, G_{5,B}, A_{6,G}, E_{8,A}$	$D_{11,E}, F_{11,E}$
$C_{0,-}, B_{3,C}, D_{2,C}, G_{5,B}, A_{6,G}, E_{8,A}, F_{11,E}$	$G_{16,F}$

Can you refer to distances and paths maps and visited set below? :D



visited	distances	paths
C, D	A: 8, B: 3, C: 0, D: 2, E: ∞, F: ∞, G: ∞	A: C, B: C, C: C, D: C, E: , F: , G:
B, G, A	A: 6, B: 3, C: 0, D: 2, E: 8, F: ∞, G: 5	A: C, B: C, C: C, D: C, E: C, F: , G: B
E	A: 6, B: 3, C: 0, D: 2, E: 8, F: 11, G: 5	A: C, B: C, C: C, D: C, E: C, F: G, G: B

- (b) Is Dijkstra's Algorithm an example of a greedy algorithm? If so, what's greedy about it? If not, to which 'family' of algorithms does it belong, and why?

Yes - Dijkstra's Algorithm is greedy because by adding to the Known/Fringe table as we traverse the graph, we assume that the optimal solution is equal to the optimal choice (lowest-weight neighbor) at that point plus an optimal sub-solution proceeding. Additionally, it is attempting to find the optimal solution to a problem (shortest path) by making many isolated locally optimal sub-solutions.

- (c) Answer the following graph terminology questions, based on the graph shown above. Either circle your answer, or write it in the space(s) provided, as appropriate.

- Which vertices are adjacent to vertex B? A, E, G
- ☒ Yes ☐ No Is this graph a simple graph? no self-loops
- Yes ☒ No ☐ Is the graph acyclic? cycle exists: {A, E, D, A}
- What is the degree of vertex A? 4
- Yes ☒ No ☐ Is this graph connected? *It is weakly connected but not strongly connected
- Yes ☒ No ☐ Is this graph complete? Every vertex is not adjacent
- Yes ☒ No ☐ Is this graph also an unrooted tree?

12. (12 points) One sorting method that we mentioned in class is TreeSort. (Reminder: TreeSort builds an ordinary BST out of the keys, and then performs an inorder traversal to get the keys in sorted order.)

(a) What is the $O()$ for TreeSort-ing n values in the best, worst, and average cases for key comparisons?

Best Case: $O(n \log_2 n)$
 Worst Case: $O(n^2)$
 Average Case: $O(n \log_2 n)$

Build: $n \log_2 n$
 insert $(\log_2 n)$ n times $\rightarrow n \log_2 n$
 Traversal: n
 worst case: every insert goes to end: $\frac{n(n+1)}{2} \in O(n^2)$

- (b) If we implement TreeSort with an AVL tree instead of a normal BST, will those best, worst, and average cases each be improved, made worse, or left unchanged? For each case, circle your answer **and** briefly explain why it is true.

(i) Best Case: Improved Made Worse Unchanged

Why?

In both cases, we insert n times into a BST — no rebalancing in AVL and all leaves on bottom level for either case — we do this n times for $O(n \log n)$. I.e., the process is the same from the perspective of asymptotic analysis.

(ii) Worst Case: Improved Made Worse Unchanged

Why?

In the case of the AVL, the inherent balance condition prevents the BST worst case of each level only having 1 node.

The rebalancing introduces its own overhead — but just considering Key comparisons, it improves to $O(n \log n)$

(iii) Average Case: Improved Made Worse Unchanged

Why?

There will be differences, but not enough differences to change the $O()$ for Key comparisons to a non-constant-factor. The average case for both will remain $O(n \log n)$.

13. (2 points) On your way home after this exam, a CSc 210 student walks up to you with a quizzical look on his face. You're feeling magnanimous, and agree to entertain the student's question: "I know what a binary heap is, but there are lots of other kinds of heaps. Why would anyone need more kinds of heaps?" You are tempted to say 'Go away, kid; you're bothering me,' but then you remember, 'Oh, right; I'm feeling magnanimous.' Besides, you now know a lot about data structures and algorithms, even though you haven't actually studied anything other than binary heaps in-depth. What is your answer, based on what you're learned in CSc 345?

The most general answer I would give is to explain that a heap can be represented with a binary tree and that by changing the implementation we can alter the general nature of the tree to better fit our needs. For example, if we use a 3-heap, one immediate consequence would be that the tree representation has become relatively squatter — thus improving search/insert/delete efficiency relatively. Then I might explain how this can be applied to specific problems like a Fibonacci heap in Dijkstra's or a heap that can be partitioned to original memory merged.